API Adopción. Equipo 7

Versión del documento	1.0
Fecha de actualización	@17 de marzo de 2026
Equipo	Ⓔ Edgar Salgado Gónzalez Angel Carlos Hérnandez Coronel Ⓐ Andrick Iorel Marcial Ahedo Juan Daniel Barrera Holan Ⓖ Gustavo Alonso Domínguez Sánchez
Descripción	**Documento evolutivo del sistema correspondiente a la Iteración 1**

Contenido

Contenido
Especificación de Requerimientos del Sistema
- 1. Introducción
  - 1.1. Propósito del documento
  - 1.2. Alcance
  - 1.3. Definiciones relevantes
- 2. Actores del sistema
- 3. Requerimientos funcionales
- 4. Casos de Uso
- 5. Requerimientos No Funcionales
- 6. Modelado de datos
  - Modelo Relacional
  - Modelo Entidad Relación

Limitaciones actuales
- 1. Tecnologías Utilizadas
- 2. Arquitectura General
  - Patrón utilizado: MVC
  - Arquitectura en Capas
- 3. Modelo Conceptual de Datos
  - Atributos Principales y Propósito
  - Consideraciones de Seguridad

Especificación de Requerimientos del Sistema

1. Introducción

1.1. Propósito del documento

El propósito de este documento es definir de manera precisa y ordenada los requerimientos del Sistema de Adopción de Mascotas, con el fin de establecer una base para su análisis, desarrollo y validación.

Estos lineamientos nos van a permitir identificar las funciones principales del sistema, los actores que van a interactuar con él y las condiciones generales que deben cumplirse para satisfacer las necesidades del proyecto. Además, servirá como referencia para el equipo de trabajo durante el desarrollo de la plataforma.

Para este documento en particular, también nos enfocaremos en las funcionalidades y requerimientos implementadas en su estado en la versión 1.0:

- En esta sección de *Especificación de Requerimientos del Sistema*, haremos un breve enfásis enfocado en las limitantes para esta versión, únicamente en las siguientes secciones: *Alcance* y *Casos de uso*.
- Profundizaremos más sobre el estado real del sistema en su versión 1.0 en *Limitaciones Actuales de la versión*.

1.2. Alcance

El sistema consiste en una plataforma orientada a la publicación y consulta de mascotas en adopción, enfocada principalmente en perros y gatos.

A través de esta plataforma, los usuarios registrados podrán crear publicaciones de mascotas disponibles para adopción, así como consultar las publicaciones existentes dentro del sistema. De igual manera, los usuarios podrán manifestar su interés en adoptar una mascota mediante un mecanismo de contacto controlado dentro de la plataforma.

El sistema busca facilitar el proceso de adopción de manera segura, evitando la exposición directa de información sensible de los usuarios, como correos electrónicos o direcciones exactas.

Alcance de la Versión 1.0

Para está versión 1.0, el alcance está centrado únicamente en funcionalidades relacionadas con la entidad Usuario. El sistema es capaz de realizar:

- Registro de usuarios
- Inicio de sesión
- Consulta de información del usuario autenticado
- Cierre de sesión

Dichas funciones se realizan de manera segura, evitando la exposición directa de información sensible a usuarios no autenticados, como nombres o correos.

1.3. Definiciones relevantes

- **ERS:** Especificación de Requerimientos del Sistema. Documento que describe las funciones, restricciones y comportamiento esperado del sistema.
- **RF:** Requerimiento Funcional. Define una función o servicio que el sistema debe proporcionar.
- **RNF:** Requerimiento No Funcional. Define restricciones o atributos de calidad del sistema, como seguridad, rendimiento o disponibilidad.
- **CU:** Caso de Uso. Describe la interacción entre un actor y el sistema para alcanzar un objetivo específico.

2. Actores del sistema

- **Actor: Usuario registrado**

Descripción: Aquella persona que ha realizado el proceso de registro dentro de la plataforma y ahora puede interactuar con todas las funcionalidades dentro de esta, este actor engloba de manera general a aquellos usuarios registrados que están interesado o interactúan con una funcionalidad especifica de la plataforma, ya sean los usuarios que publican animales en adopción, o los usuarios interesados en las publicaciones de animales en adopción.

- **Actor: Publicador de mascota**

Descripción: Usuario registrado que crea una publicación de una mascota en adopción dentro de la plataforma. Su función principal es proporcionar la información del animal y recibir notificaciones cuando otro usuario manifiesta interés en la publicación.

- **Actor: Usuario no autenticado (Visitante)**

Descripción: Es cualquier persona que accede a la plataforma web sin haber iniciado sesión. Su interacción con el sistema está altamente limitada. Solo es

capaz de visualizar la página para registrar una cuenta nueva y acceder al formulario de inicio de sesión. El sistema le bloquea cualquier intento de consulta del catálogo o publicación de mascotas.

- **Actor: API Externa (The Dog API / The Cat API)**

Descripción: Actor externo que provee servicios web utilizados por el sistema para consultar el catálogo de razas de perros y gatos, así como obtener información adicional sobre sus características, cuidados y temperamento. El sistema consume estos servicios para enriquecer la información mostrada en las publicaciones de mascotas.

- **Actor: Sistema de correo electrónico**

Descripción:

Actor externo encargado del envío automatizado de notificaciones por correo electrónico generadas por la plataforma. Este servicio permite informar al usuario publicador cuando otro usuario manifiesta interés en una mascota publicada, actuando como intermediario en la comunicación inicial entre usuarios.

3. Requerimientos funcionales

RF1: Registro de usuarios en la plataforma.

Descripción:

Cualquier persona interesada en interactuar con la plataforma deberá de realizar un proceso de registro para poder acceder a las funcionalidades del sistema. Cada persona interesada en registrarse debe proporcionar de manera mínima su nombre completo, una dirección de correo electrónico válida, además de su código postal.

Criterios de Aceptación:

- Se debe asegurar que antes de dar de alta en la plataforma a un usuario, este nos haya proporcionado su nombre completo, correo electrónico válido (importante al ser el medio de contacto con los usuarios) y su código postal.
- Si es proporcionada una contraseña para autenticar el ingreso de un usuario a la plataforma, es necesario cifrar está información antes de registrarla en nuestro sistema de almacenamiento (Alta relación con los requerimientos no funcionales de seguridad de este proyecto)

RF2: Publicación de animales en adopción.

Descripción:

Cualquier usuario registrado tiene la capacidad de publicar una publicación de un animal (perro o gato) en adopción dentro de la plataforma. Cada usuario registrado interesado en está funcionalidad deberá de proporcionar el nombre del animal, una descripción física breve del animal, submitir al menos una fotografía del animal en cuestión, además del tipo (perro o gato), su raza o similar, además del código postal donde reside la mascota para poder realizar la publicación.

Criterios de Aceptación:

- Para poder realizar la publicación en la plataforma, es necesario proporcionar **al menos** la siguiente información: Nombre, descripción física, fotografías, tipo, raza y código postal del animal para realizar la publicación de adopción.
- Al deber contar con una descripción general del animal para realizar una publicación, se debe asegurar que la descripción tenga un tamaño razonable para ser almacenado y ser lo suficientemente descriptivo sin ser tan grande como para consumir más recursos de almacenamiento.

RF3: Consulta de publicaciones de mascotas disponibles

Descripción:

El sistema debe permitir que el usuario registrado consulte las publicaciones de mascotas disponibles para adopción, mostrando la información principal de cada una, como nombre, tipo, raza, descripción, fotografía y código postal.

Criterios de Aceptación:

- El sistema debe mostrar las publicaciones activas de mascotas disponibles para adopción.
- Cada publicación debe incluir la información principal de la mascota.
- El usuario debe poder visualizar la información sin necesidad de modificar la publicación.
- Solo deben mostrarse publicaciones que se encuentren disponibles.

RF4: Manifestación de interés en una publicación

Descripción:

El sistema debe permitir que el usuario registrado manifieste su interés en adoptar una mascota mediante la opción “Me interesa” dentro de la

publicación. Al realizar esta acción, el sistema deberá registrar la solicitud y generar una notificación para el usuario que publicó la mascota.

Criterios de Aceptación:

- El usuario debe haber iniciado sesión para usar la opción “Me interesa”.
- El sistema debe registrar la solicitud de interés asociada a la publicación seleccionada.
- El sistema debe notificar al publicador de la mascota que existe un usuario interesado.
- Si la publicación ya no está disponible, el sistema debe impedir la solicitud y mostrar un mensaje informativo.

RF5: Visualización geoespacial segura de mascotas

Descripción:

El sistema deberá proveer una interfaz de mapa interactivo, en el que muestre a los usuarios registrados las mascotas disponibles, agrupándolas exclusivamente por su código postal para proteger su ubicación exacta.

Criterios de Aceptación:

- El mapa deberá mostrar un marcador en aquellos códigos postales que cuenten con al menos un registro de mascota y que esté disponible para adopción.
- Los marcadores en el mapa deberán corresponder a un punto central aproximado del código postal correspondiente (un sólo marcador por cada código postal).
- Al seleccionar un marcador, el sistema deberá mostrar la lista de mascotas disponibles (junto con el enlace a su información) ubicadas en ese código postal.
- El usuario podrá navegar manualmente arrastrando el mapa.

RF6: Control de acceso al sistema

Descripción:

El sistema deberá restringir el acceso a las funciones de consulta y publicación de mascotas solo a usuarios registrados, exigiendo su autenticación previa para el uso de la plataforma.

Criterios de Aceptación:

- Todo usuario no autenticado podrá acceder a la página de autenticación (funciones de registro / inicio de sesión).
- Todo usuario autenticado será redirigido a la página principal (funciones de consulta y publicación habilitadas) si intenta acceder a la página de autenticación.
- Cuando un usuario no autenticado intenta acceder a una ruta protegida (como el de consulta o publicación), el sistema denegará su acceso y lo redirigirá automaticamente a la página de autenticación con un mensaje indicando que la autenticación es requerida.
- Una vez que el usuario ingrese credenciales válidas, el sistema deberá otorgarle acceso inmediato y redirigirlo a la página principal (con las funciones de consulta y publicación).
- Si el usuario ingresa credenciales inválidas, permanece en la página de autenticación con el formulario limpio y un mensaje de credenciales erróneas.

RF7: Integración con API externa para selección y detalle de raza

Descripción:

El sistema deberá integrar las APIs públicas de "The Dog API" y "The Cat API" para enriquecer la información de las mascotas.

Criterios de Aceptación:

- Al momento de registrar un animal, el sistema deberá permitir seleccionar la raza a partir de las opciones disponibles en la API correspondiente.
- En la ficha informativa del animal, el sistema deberá mostrar una sección adicional con datos obtenidos de la API.
- Esta sección adicional deberá contener información general de la raza, recomendaciones de cuidado y características relevantes como el temperamento o nivel de actividad.

RF8: Notificación automatizada y segura por correo electrónico

Descripción:

El sistema deberá actuar como intermediario en el primer contacto, enviando automáticamente un correo electrónico al publicador cuando alguien pulse el botón "Me interesa".

Criterios de Aceptación:

- El correo generado por el sistema deberá incluir obligatoriamente al menos un dato de contacto del usuario interesado, como su nombre y su correo electrónico.
- Bajo ninguna circunstancia el sistema mostrará directamente los correos electrónicos de los usuarios en la interfaz gráfica web.

4. Casos de Uso

Adecuaremos el siguiente caso de uso considerando el alcance del sistema en esta versión, por lo que otros casos de uso que estén fuera de este alcance, se analizarán en versiones posteriores.

CU3: Autenticación de usuarios y acceso a funcionalidades

Actores: Usuario no autenticado

Precondiciones:

- El usuario deberá contar previamente con credenciales válidas en la base de datos.
- El usuario no se ha autenticado aún.

Flujo Principal:

- El usuario intenra accede a cualquier pagina protegida (como /me) de la plataforma.
- El sistema detecta que el usuario no se ha autenticado.
- El sistema avisa un error. El usuario no podrá acceder.
- El usuario ingresa su correo electrónico y su contraseña válidos.
- El sistema valida las credenciales en la base de datos.
- Si las credenciales son validas, el sistema autentica al usuario.
- El usuario ahora puede visualizar su información.

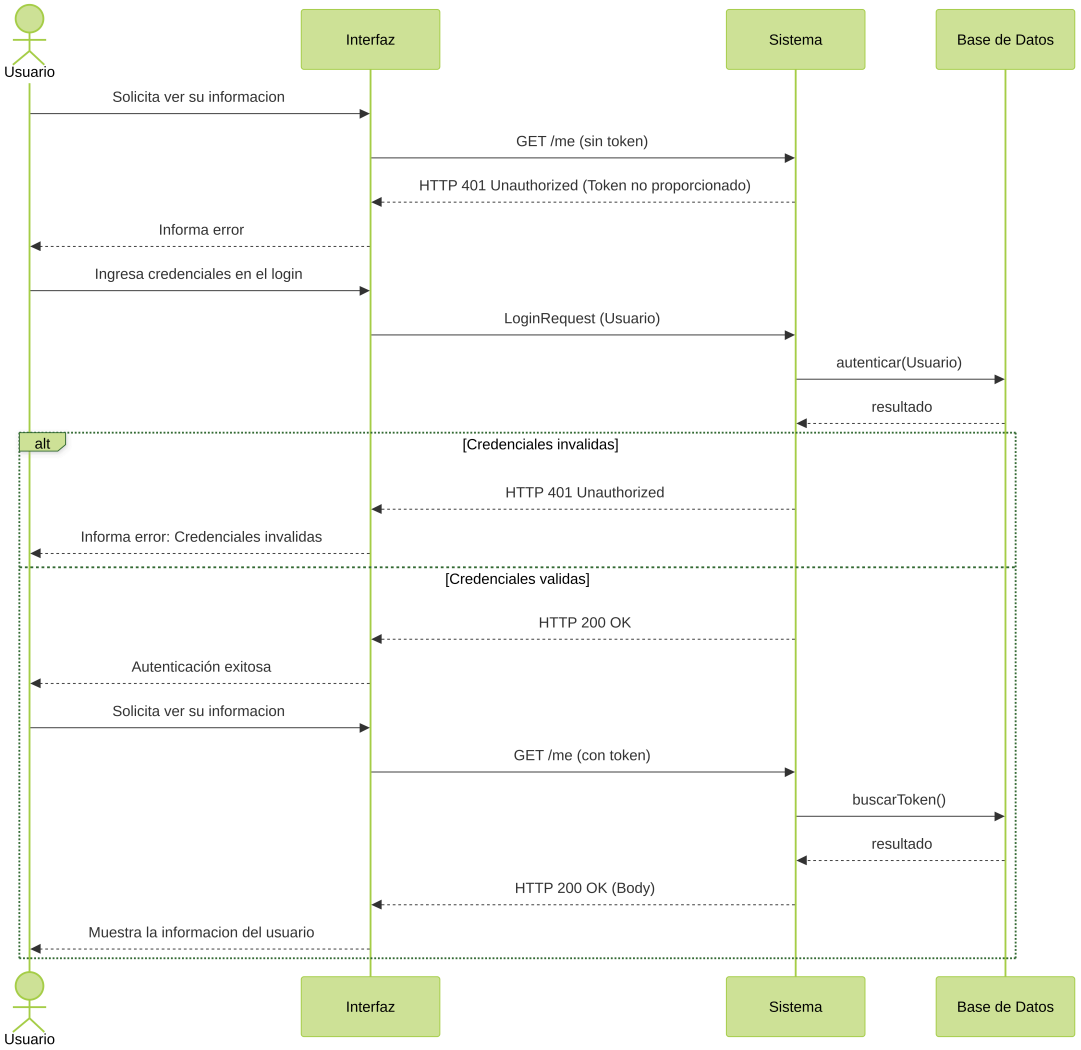
Flujo Alternativo:

Si las credenciales son incorrectas, el sistema infroma un error e informa que no se encontró al usuario.

Postcondición:

El usuario obtiene acceso al sistema y puede visualizar su información.

Diagrama de secuencia:



5. Requerimientos No Funcionales

Los siguientes requerimientos no funcionales describen las características de calidad, seguridad, rendimiento y disponibilidad que deberá cumplir la plataforma de adopción de mascotas.

RNF1: Seguridad (Protección de credenciales)

Las contraseñas de los usuarios deberán almacenarse utilizando un mecanismo de hash seguro (por ejemplo bcrypt o equivalente) y nunca deberán guardarse en texto plano dentro del sistema de almacenamiento como se representa en el diagrama en el punto número 6.

Este requerimiento asegura la protección de las credenciales de los usuarios en caso de acceso no autorizado a la base de datos.

RNF2 : Privacidad de la información

El sistema deberá proteger la información personal de los usuarios, evitando la exposición directa de datos sensibles como direcciones exactas o correos electrónicos durante la interacción entre usuarios.

La ubicación de los animales deberá mostrarse únicamente mediante referencias aproximadas basadas en el código postal.

RNF3 : Rendimiento del sistema

La carga de las publicaciones de mascotas disponibles, así como su visualización en el mapa interactivo, deberá completarse en un tiempo promedio no mayor a 3 segundos bajo condiciones normales de uso.

Este requerimiento busca garantizar una experiencia de navegación fluida para los usuarios de la plataforma.

RNF4 : Usabilidad

La interfaz de la plataforma deberá permitir que un usuario registrado pueda navegar por las publicaciones de mascotas, consultar la información de cada animal y manifestar su interés en adopción de forma intuitiva y comprensible, sin necesidad de capacitación previa.

RNF5 : Disponibilidad del sistema

La plataforma deberá mantenerse disponible al menos el 95% del tiempo durante su periodo de operación, permitiendo el acceso continuo a las funciones principales del sistema.

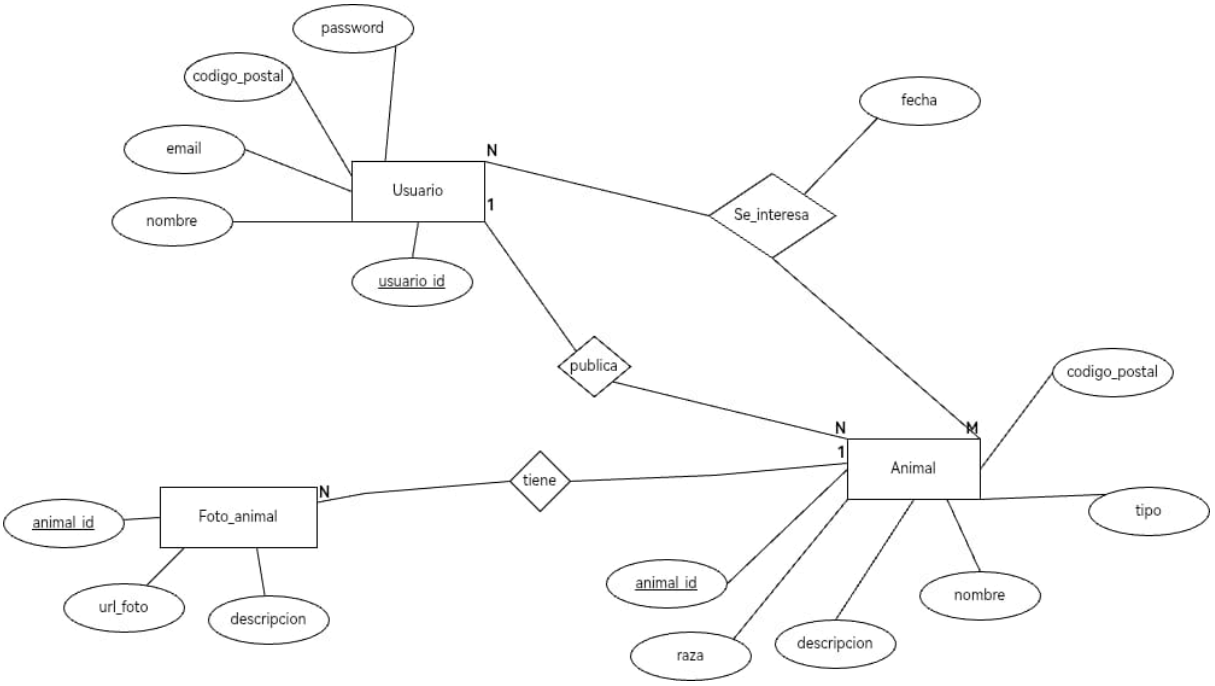
RNF6 : Integridad de los datos

El sistema deberá garantizar la consistencia e integridad de la información almacenada, evitando registros incompletos o inconsistentes en las publicaciones de animales y en las solicitudes de interés generadas por los usuarios.

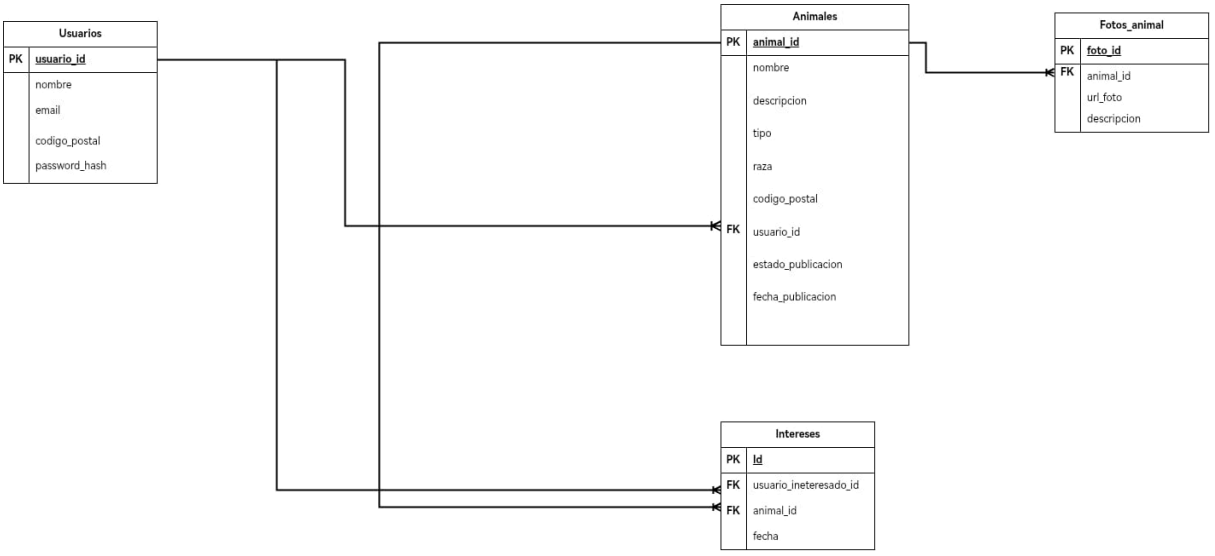
6. Modelado de datos

A la fecha de la entrega, hemos realizado está modelado de datos para conseguir los objetivos de este proyecto. Estos modelos están sujetos a cambios conforme el desarrollo de este trabajo avance.

Modelo Relacional



Modelo Entidad Relación



Limitaciones actuales

1. Tecnologías Utilizadas

- **Lenguaje de Programación**

Al momento de trabajar en está iteración, el lenguaje de programación principal ha sido Kotlin, tanto para trabajar en el backend del sistema de forma general. Este lenguaje ha demostrado ser una herramienta muy poderosa y útil para la realización de este proyecto al poder acoplarse perfectamente a las necesidades

de desarrollo, sobre todo en la implementación de la arquitectura por capas, permitiendo utilizar diversas funciones de Servicios y Repositorio como propiedades de un objeto, como es el caso de la creación de una instancia de UsuarioService en la clase de Controllers. No utilizamos kotlin exclusivamente, ya que se emplean bibliotecas de otros lenguajes para facilitarnos el trabajo, tales como Java; siendo otra ventaja de Kotlin el fácil acoplamiento de otros lenguajes en el proyecto.

- **Framework**

Para la realización de este proyecto se emplea spring framework, esto debido a su orientación al diseño de programas y la facilidad que otorga para el manejo de datos, esto es uno de los aspecto clave en esta parte tan temprana del desarrollo al tener que diseñar el registro y autenticación de usuarios.

- **Base de datos**

De manera inicial, se ha decidido utilizar el sistema manejador de base de datos (SMBD) PostgreSQL al ser puesto a discusión en nuestro equipo. El motivo de está decisión fue lo familiarizado que está el equipo con esta herramienta y sus operaciones, lo que ha facilitado la implementación en el sistema sin la necesidad de un periodo de aprendizaje que pudiera entorpecer el desarrollo.

- **Herramientas relevantes utilizadas en el desarrollo**

Además de las herramientas previamente mencionadas, se han empleado bibliotecas de otros lenguajes, principalmente de Java, las cuáles proveen elementos básicos de seguridad que se implementaron en el almacenamiento de contraseñas de usuario, se empleó específicamente MessageDigest para hacer un hash básico de las contraseñas antes de almacenarlas. Además de SLF4J, una facade de registros simple que usamos en nuestra aplicación para llevar las operaciones hasta el momento sencillas de nuestro sistema.

2. Arquitectura General

Patrón utilizado: MVC

El modelo vista-controlador (MVC) adaptado para trabajar en una API REST (MVC adaptado a REST), de manera tradicional el modelo vista-controlador se usa para aplicaciones con interfaz gráfica, pero en el caso de nuestra API, es muy útil para fragmentar la funcionalidad del sistema en 3 módulos principales, tradicionalmente Model, Visual and Controller que para está ocasión fueron

cambiados por Controller, Service y Repository que ahora fueron denominadas como “capas”.

Arquitectura en Capas

- **Controller:** En este caso, la capa de controller se encarga de recibir y manejar las solicitudes de información que entran a nuestro sistema por medio de los llamados endpoints, siendo el encargado de llamar a la capa de servicio para procesar la solicitud y en dado caso, devolver la información correspondiente.
- **Service:** La capa de servicio se encarga de la lógica interna y el proceso para llevar a cabo lo requerido por parte de las solicitudes de Controller, generalmente haciéndose aquí los procesos de registro, modificación o autenticación de datos antes de devolver la información a la capa de controllers y/o notificar los cambios a la capa repository, de la cual service extrae la información para llevar a cabo los procesos pertinentes.
- **Repository:** La capa Repository actúa como una forma de proporcionar la información requerida por las demás capas, siendo una conección con los registros más que con la interacción o modificación de estos, siendo nuestra fuente de información la cual debe permanecer fuera de operaciones directas sobre esta que no sean meras consultas o actualizaciones.

3. Modelo Conceptual de Datos

En esta primera iteración, el modelo de datos se centra exclusivamente en la entidad Usuario, la cual es el núcleo para el control de acceso y la seguridad de la plataforma.

Atributos Principales y Propósito

- **Identificador Único (ID):** Cadena alfanumérica (UUID) que garantiza la distinción inequívoca de cada usuario dentro del sistema, evitando colisiones de datos.
- **Nombre:** Almacena la identidad del usuario para personalizar la experiencia en la plataforma y facilitar el contacto humano en el proceso de adopción.
- **Correo Electrónico (Email):** Funciona como el identificador de acceso (login) y el canal de comunicación oficial y seguro entre el adoptante y el publicador.
- **Código Postal (CP):** Atributo estratégico para la visualización geoespacial segura. Permite agrupar las mascotas en un mapa por zonas sin vulnerar la

privacidad del usuario al no exponer su dirección exacta.

- **Contraseña (Password):** Credencial indispensable para validar la identidad del usuario y otorgar acceso a las funcionalidades restringidas del sistema.

Consideraciones de Seguridad

Para cumplir con los lineamientos de seguridad y privacidad del proyecto:

- **Cifrado de Credenciales:** Bajo ninguna circunstancia se almacenan contraseñas en texto plano. El sistema utiliza un mecanismo de hashing SHA-256 (procesado en la capa de lógica de negocio) antes de persistir la información en la base de datos.
- **Integridad de Datos:** El correo electrónico se define como un campo único y obligatorio, asegurando que no existan cuentas duplicadas y que siempre exista un medio de contacto válido.
- **Protección de Privacidad:** El modelo está diseñado para que la ubicación solo sea referencial (vía código postal), cumpliendo con el principio de no exposición de información sensible.

Durante el inicio de sesión, el sistema no compara directamente la contraseña en texto plano contra la base de datos. En su lugar, toma la contraseña proporcionada en el request, le aplica el mismo proceso de hash y compara el resultado con el valor almacenado. Esto permite autenticar al usuario sin necesidad de recuperar ni exponer la contraseña original.

Adicionalmente, se realizaron ajustes en las respuestas de los endpoints relacionados con usuarios para evitar exponer información sensible al cliente. Aunque internamente el objeto Usuario contiene el atributo password, en las respuestas HTTP se devuelve una versión sanitizada del objeto, estableciendo dicho atributo en null. Esta decisión se aplicó en operaciones como registro, consulta de usuario y actualización, con el propósito de impedir que la contraseña o incluso su hash sean visibles desde la API.

Capa Service para registro e inicio de sesión

Durante el inicio de sesión, el sistema no compara directamente la contraseña en texto plano contra la base de datos. En su lugar, toma la contraseña proporcionada en el request, le aplica el mismo proceso de hash y compara el resultado con el valor almacenado. Esto permite autenticar al usuario sin necesidad de recuperar ni exponer la contraseña original.

Adicionalmente, se realizaron ajustes en las respuestas de los endpoints relacionados con usuarios para evitar exponer información sensible al cliente. Aunque internamente el objeto Usuario contiene el atributo password, en las respuestas HTTP se devuelve una versión sanitizada del objeto, estableciendo dicho atributo en null. Esta decisión se aplicó en operaciones como registro, consulta de usuario y actualización, con el propósito de impedir que la contraseña o incluso su hash sean visibles desde la API.

En el proceso de registro, el controlador recibe un objeto CreateUsuarioRequest, lo transforma a un objeto de dominio Usuario y delega al servicio la operación de alta. Dentro de UsuarioService, la función addNewUsuario verifica primero si ya existe un usuario con el mismo correo electrónico. Si el correo ya se encuentra registrado, el servicio devuelve null, permitiendo que el controlador responda con un conflicto (HTTP 409). Si no existe duplicidad, el servicio aplica hash a la contraseña y posteriormente guarda el usuario en la base de datos mediante el repositorio.

En el proceso de inicio de sesión, el controlador recibe un LoginRequest con correo y contraseña. La función authenticate del servicio busca al usuario por correo mediante findByEmail. Si el usuario no existe, la autenticación falla de inmediato. Si existe, el servicio vuelve a calcular el hash de la contraseña proporcionada y compara el resultado con el valor almacenado en la base de datos. Solo cuando ambos valores coinciden se considera autenticación exitosa.

También se implementó la lógica de actualización de usuario mediante la función updateUsuario. Esta operación permite modificar nombre, correo, código postal y, de forma opcional, la contraseña. En caso de que se reciba una nueva contraseña, el servicio aplica nuevamente el hash antes de guardar el cambio, manteniendo la misma política de seguridad definida para el registro

Durante el desarrollo se detectó que algunos endpoints devolvían directamente el objeto Usuario tal como se encontraba en memoria. Esto implicaba el riesgo de exponer el campo password en la respuesta del API, ya fuera en texto plano o como hash. Para corregir este comportamiento, se modificaron los endpoints para devolver una versión sanitizada del objeto de usuario, estableciendo el atributo password en null antes de enviarlo al cliente. Esta modificación se aplicó en las operaciones de registro, consulta y actualización.

Como consecuencia de este ajuste, se cambió la firma de algunos métodos del controlador de `ResponseEntity<Usuario>` a `ResponseEntity<Any>`. La razón es que estos endpoints ya no siempre devuelven exactamente el mismo tipo de respuesta: en algunos casos regresan un usuario sanitizado, mientras que en otros devuelven mapas con mensajes de error o respuestas vacías asociadas a distintos códigos

HTTP. El uso de `ResponseEntity<Any>` permite reflejar correctamente este comportamiento heterogéneo sin forzar una firma que no coincida con la lógica real del endpoint.

Implementación inicial de autenticación basada en Tokens

Al final del desarrollo de esta primera iteración, se identificó que cumplir estrictamente con el registro y el inicio de sesión no era suficiente para garantizar la seguridad de los usuarios a largo plazo. Para consultar el perfil (el endpoint `/me`) o cerrar sesión, el sistema necesitaba una forma segura de saber "quién" estaba haciendo la petición, sin obligar al usuario a enviar su correo y contraseña cada vez.

Por ello, aunque no se pide explicitamente implementar autenticación, se tomó la decisión de que necesitabamos implementar un sistema inicial de autenticación basado en Tokens. Con esto podemos aseguraramos de que nuestro sistema tiene:

- **Privacidad de datos:** Evita que cualquier persona pueda consultar la información de otro usuario.
- **Sesiones seguras:** Permite que el sistema "recuerde" que un usuario ya inició sesión de forma segura.

La forma en la que se integró manteniendo la limpieza de nuestra arquitectura en capas:

1. **Generación del Token:** Cuando un usuario realiza un Login exitoso validando su contraseña cifrada, la capa de lógica (`Service`) genera un identificador único universal (UUID).
2. **Persistencia:** Este token temporal se guarda en la base de datos (PostgreSQL) vinculado a la entidad de ese usuario específico.
3. **Uso en la API:** El token se devuelve al cliente y, a partir de ese momento, el cliente lo mantiene oculto en las cabeceras HTTP (`Headers: Authorization`) para cualquier acción privada.
4. **Cierre de sesión:** Al realizar un endpoint de *Logout*, el sistema busca el token activo y lo elimina (se establece como nulo en la base de datos), invalidando la sesión de forma inmediata.